Theoretical Computer Science 412 (2011) 4814–4823

Contents lists available at ScienceDirect

Theoretical Computer Science

journal homepage: www.elsevier.com/locate/tcs

Computing efficiently the lattice width in any dimension

Émilie Charrier [a,*], Fabien Feschet [b], Lilian Buzer [c]

[a] Université de Savoie, Laboratoire de Mathématiques, UMR 5127 CNRS, F-73376 Le Bourget du Lac, France
[b] Clermont Université, Université d'Auvergne, Santé-Technologie, 49 Bd F. Mitterrand, BP 32, F-63001 Clermont-Ferrand, France
[c] Université Paris-Est, LABINFO-IGM, UMR CNRS 8049, ESIEE Paris, 2 Bd Blaise Pascal, BP 99, F-93162 Noisy le Grand Cedex, France

ARTICLE INFO

Keywords:
Lattice width
Shortest vector
Greedy algorithm

ABSTRACT

We provide an algorithm for the exact computation of the lattice width of a set of points K in $\mathbb{Z}^2$ in linear-time with respect to the size of K. This method consists in computing a particular surrounding polygon. From this polygon, we deduce a set of candidate vectors allowing the computation of the lattice width. Moreover, we describe how this new algorithm can be extended to an arbitrary dimension thanks to a greedy and practical approach to compute a surrounding polytope. Indeed, this last computation is very efficient in practice as it processes only a few linear time iterations whatever the size of the set of points. Hence, it avoids complex geometric processings.

© 2011 Elsevier B.V. All rights reserved.

1. Introduction

Integer Linear Programming is a fundamental tool in optimization, in operational research and in economics.... Moreover, it is interesting in itself since the problem is NP-hard in the general case. Several works were done for the planar case [20,28,15] before Lenstra [23] proved that Integer Linear Programming can be solved in polynomial time when the dimension is fixed. Faster and faster algorithms are nowadays developed and available, making the use of Integer Linear Programming reliable even for high dimensional problems. The approach of Lenstra uses the notion of *lattice width* for precise lattice definition to detect directions for which the polyhedron of solutions is thin. In polynomial time, the problem is then reduced to a feasibility question: given a polyhedron P, determine whether P contains an integer point. To solve it, Lenstra approximates the width of the polyhedron and gives a recursive solution solving problems of smaller dimension. The approximate lattice width is also used in the recent algorithms of Eisenbrand and Rote [10] and Eisenbrand and Laue [9] for the 2-variable problem.

Not surprisingly, following the arithmetical approach of Reveillès [26,7], the lattice width is also a fundamental tool in digital geometry since it corresponds to the notion of width for digital objects [11]. Moreover, as an application of the lattice width computation, we mention the intrinsic characterization of linear structures [12]. Indeed, the lattice width can be computed for any digital set but it does not correspond to a direct measure of linearity. However, when combining the lattice width along a direction and along its orthogonal, it can be used as a linearity measure. The work in [12] is currently extended, by the second author of the present paper, to higher dimensions for detecting either linear or tubular structures. A preliminary algorithm for the computation of the lattice width in the two-dimensional case was given in [11] with a geometrical interpretation. It has the advantage to be extensible to the incremental and to the dynamic case but it seems difficult to extend it to an arbitrary dimension. We proposed in [4] a new method, efficient in any dimension, and we extend it here by detailing its main steps and by providing experimental results. This approach is based on the computation of a particular surrounding polytope which is used to bound the set of candidate vectors to define the lattice width. This

* Corresponding author. Tel.: +33 479759438; fax: +33 479758142.
E-mail addresses: Emilie.Charrier@univ-savoie.fr (É. Charrier), feschet@acm.org (F. Feschet), buzerl@esiee.fr (L. Buzer).

0304-3975/$ – see front matter © 2011 Elsevier B.V. All rights reserved.
doi:10.1016/j.tcs.2011.02.009

algorithm runs in linear time for the two-dimensional case. In higher dimensions, we propose a greedy method to compute the surrounding polytope. This approach is efficient as it computes a maximal simplex over the set of points in few iterations whatever the size of the set of points. Then, we directly deduce an appropriate surrounding polytope from this simplex.

The paper is organized as follows. In Section 2, the main definitions and tools are presented. Then, we describe the two-dimensional algorithm introduced in [11] in Section 3. As this method cannot be easily extensible to higher dimensions, we introduce in Section 4 a new geometric approach to estimate the lattice width in any dimension. This algorithm is based on the computation of a particular surrounding polytope and we describe in Section 5 geometric methods to compute it. We first focus on the two-dimensional case and we provide two linear-time approaches. Then, since these geometric constructions might be difficult to extend in arbitrary dimensions, we provide a greedy algorithm which runs in any dimension. Some conclusions and perspectives end the paper.

2. Definitions from integer lattice theory

In this section, we review some definitions from algorithmic number theory and we provide a precise formulation of the problem we solve. Definitions are taken from [2,10,29].

Let K be a set of n points of $\mathbb{Z}^d$. Moreover, we suppose that all numbers appearing in the points and in the vector coordinates have their bit size bounded by $\log s$. The *width* of K along a direction $c \neq 0$ in $\mathbb{R}^d$ is defined as:

$$\omega_c(K) = \max\left\{c^T x \mid x \in K\right\} - \min\left\{c^T x \mid x \in K\right\}. \tag{1}$$

Geometrically, if a set K has a width of l along the direction c then any integer point which lies in the interior of the convex hull of K also lies on a hyperplane of the form $c^T x = \lambda$ where λ corresponds to an integer value between $\min\{c^T x \mid x \in K\}$ and $\max\{c^T x \mid x \in K\}$. We say that K can be covered by these $\lfloor l \rfloor + 1$ parallel hyperplanes. It is straightforward to see that $\omega(K) = \omega(\text{conv}(K))$ where $\text{conv}(K)$ denotes the convex hull of K.

Let $\mathbb{Z}^{d*} = \mathbb{Z}^d \setminus \{0\}$ denote the set of integer vectors different from zero. The *lattice width* of K is defined as follows:

$$\omega(K) = \min_{c \in \mathbb{Z}^{d*}} \omega_c(K). \tag{2}$$

We notice that the lattice width is an integer value. We briefly recall some basic and important properties about inclusion and translation:

Lemma 1. *For any sets of points A and B, such that conv(A) ⊂ conv(B) and for any vector $c \in \mathbb{Z}^{d*}$, we have $\omega_c(A) \leq \omega_c(B)$. Thus, it follows that $\omega(A) \leq \omega(B)$.*

Lemma 2. *Suppose that A' corresponds to the points of A translated in the same direction. By definition, we know that for any $c \in \mathbb{Z}^{d*}$, $\omega_c(A) = \omega_c(A')$ and so we have $\omega(A) = \omega(A')$. The lattice width is invariant under translation.*

The problem we would like to solve is the following one:

Problem (Lattice Width)
Given a set of integer points $K \subset \mathbb{Z}^d$, find its lattice width $\omega(K)$ as well as all the vectors $c \in \mathbb{Z}^d$ such that $\omega_c(K) = \omega(K)$.

It is known [23] that the lattice width of a convex set K is obtained for the shortest vector with respect to the *dual norm* whose unit ball is the polar set of the set $\frac{1}{2}(K + (-K))$. In the general case, computing the shortest vector is NP-hard. Thus, approximations of the solution can be computed via standard arguments [29,19,27], but it does not lead us to an easy exact algorithm in arbitrary dimension.

3. Computing lattice width in the planar case

3.1. The 2006 algorithm design

In 2006, Feschet proposed in [11] a method to compute in $O(n + n \log s)$ time the lattice width of a set of two-dimensional integer points. We recall in this part this algorithm via connections with the notion of digital straightness and more precisely with the notion of arithmetical digital lines [26]. This two-dimensional algorithm requires a convex polygon as input; as a consequence we have to compute the convex hull H of K in $O(n)$ time [18].

The main idea in [11] is based on the principle that the lattice width of K is necessarily reached for two *opposite* vertices of its convex hull. To define the notion of *opposite*, we rely on the notion of *supporting lines* well known in computational geometry [6]. A *supporting line* of H is a line D such that $D \cap K \neq \emptyset$ and H is contained entirely in one of the half-planes bounded by D. For each supporting line D, there exists at least one vertex v of H such that the parallel line D_v to D passing through v is such that H entirely lies in the strip bounded by D and D_v. If s denotes a vertex of H belonging to D then s and v are called *opposite* (see Fig. 1, left). Opposite pairs are also called *antipodal* pairs. Note that in general, a supporting line intersects H at only one point. The supporting line D intersecting H along an edge is called *principal* supporting line.

We now suppose H to be oriented counter-clockwise. As in the classical Rotating Calipers algorithm of Toussaint [16], we can rotate the principal supporting lines D around the right vertex of $D \cap H$. D_v is also rotated around v to keep it parallel

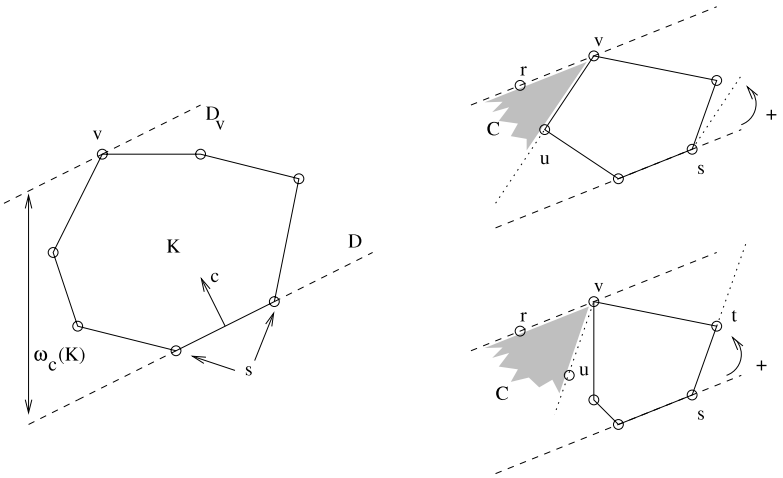


Fig. 1. (left) Supporting lines and width $\omega_c(K)$; (right) Cone of rotations.

to D. This rotation can be pursued until D or D_v corresponds to another principal supporting line. Note that D and D_v are simply supporting lines during the rotation. At each position of the rotation s and v form an opposite pair of points which exactly define $\omega_c(H)$ where c is the normal direction to D. Hence, as depicted in Fig. 1 (right), s and v exactly define $\omega_c(H)$ for all D in a pointed cone whose apex is v. The point r is such that the segment from v to r has exactly the same length than the opposite edge of H and the point u is either the next vertex of H after v or the point on the parallel of the line (st) such that the length of $[st]$ equals the length of $[vu]$.

After one turn around H, we have constructed at most $2n$ opposite pairs and associated cones. Hence, the number of cones is $O(n)$. Moreover, the series of cones forms a partition of all possible directions of computation for the lattice width taking into account that $\omega_c(H) = \omega_{-c}(H)$. Hence, as previously announced, the computation of the lattice width is reduced to the computation of the minimal value of $\omega_c(H)$ for each cone.

In each cone C, the computation of the minimal value of $\omega_c(H)$ is the computation of the shortest vectors for the dual norm. They are thus located at the vertices of the border of the convex hull of integer points except v inside the cone [19]. This set is also known as Klein's sail [21,22,1]. Note that to allow the possibility to find all solution vectors, repetitions in the convex hull must be kept. To compute Klein's sail, we use an adapted version of the algorithm of Harvey [14] whose complexity is $O(\log s)$ arithmetic operations for each sail computation. To bound the complexity of the search, we could also rely on the general theorem of Cook et al. [5] which says that there exists at most $O((\log s)^{d-1})$ vertices in dimension d, a result also shown by Harvey [14] with an explicit example of the worst case in two dimensions. Thus, the complexity of the lattice width computation is $O(n + n \log s)$.

3.2. Drawbacks of the method

The main drawback of this algorithm is that it is intrinsically a planar algorithm. Indeed, the geometric concepts can be extended to an arbitrary dimension d and it is really possible to walk on the convex hull and to determine the opposite vertex. But, even in 3d, this does not seem to be a trivial task to generalize the cone of rotations. Indeed, it is classical [17] that the case of vertex-face is not the only case of opposing structures but that two edges might be opposite and determine the geometric width of the set. Indeed, the cone of rotations is no more a pointed cone in general. As such the d-dimensional extension of Klein's sails is not sufficient to extend our algorithm. Computing the convex hull of the integer points inside a non-pointed cone is a lot more complicated than in the planar case. Hence, we did not work in this way and we have chosen a rather new strategy that we present in the next section.

4. A different approach to compute lattice width in any dimension

The geometric approach of the previous algorithm appears as a major drawback for an arbitrary dimension. Hence, we propose a different geometric method based on the possibility of bounding the set of candidate directions used to compute the lattice width in the d-dimensional case. Previously introduced in [4], we detail the principle of the algorithm.

4.1. Overview of the method

Let K denote a set of d-dimensional integer points. Starting with the definition of the width, with c a vector in $\mathbb{Z}^{d*}$, we know that:

$$\omega_c(K) = \max_{u \in K}\{c^T u\} - \min_{u \in K}\{c^T u\} = \max_{u,v \in K}\{c^T (u - v)\}.$$

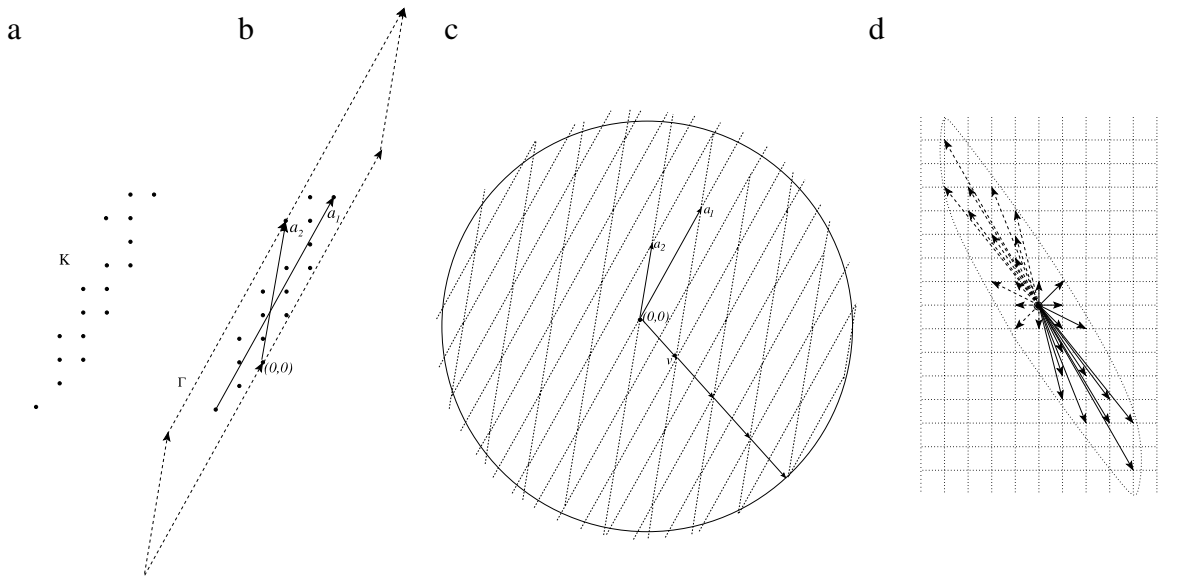


Fig. 2. Initial set of points K (a), surrounding polygon Γ (b), lattice generated by a_1 and a_2, shortest vector v and ball of radius $4\|v\|$ (c) and set of candidate vectors corresponding to the irreducible vectors included in the ball transposed in the orthogonal lattice (d).

Hence, the width along the direction c is obtained for a vector $a = u - v$ where both u and v are vertices of K. Such a vector is called a *chord* of the set K. Since the lattice width is reached for some directions c_w, we conclude that the good directions should be built upon the chords of K. To obtain an efficient algorithm, we must forbid one to take into account the whole set of chords (the set $K - K$). Our construction is based on the fact that if we have a convex set Γ which contains K and which is solely based on a well chosen subset of chords then Γ will provide us with a set of candidate directions containing the optimal directions for K. Then, an enumeration of that set is sufficient to obtain the lattice width of K.

The method consists in three main steps. While we describe the algorithm in the d-dimensional case, we illustrate it by an example in the planar case (see Fig. 2). Let K denote the initial set of points in $\mathbb{Z}^d$ (see Fig. 2a); the three steps are:

1. Computing a surrounding polytope Γ of K whose vertices γ_i are expressed as an integer linear combination of d chords $(a_j)_{1\leq j\leq d}$ of K, that is $\gamma_i = \sum_{1\leq j\leq d} \alpha_{ij} a_j$. We also have $|\alpha_{ij}| \leq \alpha$ where α denotes a constant, in the example α is equal to 1 (Fig. 2b)
2. Deducing from the lattice induced by the $(a_j)_{1\leq j\leq d}$ the set of associated candidate vectors. For this:
   (a) Computing the shortest vector v in the polytope lattice and generating a ball of radius $2\alpha d\|v\|$. In the two-dimensional case, the radius is equal to $4\alpha\|v\|$ (Fig. 2c).
   (b) Selecting all the vectors which can be written as an integer linear combination of the $(a_j)_{1\leq j\leq d}$ and which are included in the ball. Transforming them into the orthonormal lattice, they exactly correspond to the candidate vectors (Fig. 2d).
3. Computing the width of the set K relative to each candidate vector and selecting the minimum width which correspond to the lattice width.

In the two-dimensional example, we can compute 13 different candidate vectors taking into account that $\omega_c(K) = \omega_{-c}(K)$ and considering only irreducible vectors. The set of candidate vectors corresponds to $\{(0, 1), (1, 1), (1, 0), (2, -1), (1, -1), (4, -5), (3, -4), (3, -5), (4, -7), (1, -2), (2, -5), (1, -3), (1, -4)\}$.

The lattice width of the set of points is equal to 4 and it is achieved for the candidate vector $(2, -1)$.

4.2. Core idea and mathematical results

Let us now describe our new algorithm more formally in the d-dimensional case.

Let K denote a set of d-dimensional integer points. Let $(a_j)_{1\leq j\leq d}$ denote a sequence of d chords in $\mathbb{Z}^d$, i.e. such that for any j, $1 \leq j \leq d$, there exist two points u_j and v_j of K satisfying $a_j = u_j - v_j$. Let us consider a surrounding polytope $\Gamma = \text{conv}((\gamma_i)_{1\leq i\leq m})$ such that Γ contains $\text{conv}(K)$ and such that each of its vertices γ_i, $1 \leq i \leq m$, satisfies $\gamma_i = p + \sum_{1\leq j\leq d} \alpha_{ij} a_j$. We present our core idea. If we can compute a surrounding polytope such that the values $|\alpha_{ij}|$, $1 \leq i \leq m$, $1 \leq j \leq d$, are bounded by a constant α, then we can determine a set of candidate vectors C satisfying these properties: the cardinality of C is bounded by a constant and the set C contains all the solution vectors of our lattice width problem. Thus, by computing all the $\omega_u(K)$, $u \in C$, in $O(dn)$ time, we determine $\omega(K)$ and the associated solution vectors. Moreover, the constant which bounds the cardinality of C is independent from K and depends only on d and α. We postpone the construction of such a set Γ until the next section and suppose here that we are able to get it from K to focus on the mathematical results.

Let c denote a vector in $\mathbb{Z}^{d*}$. By the definition of width, we know that:

$$\omega_c(\Gamma) = \max_{u\in\Gamma}\{c^T u\} - \min_{v\in\Gamma}\{c^T v\}.$$

As the lattice width is translation-invariant, we obtain the following inequality:

$$\omega_c(\Gamma) \leq 2 \max_{1\leq i\leq m} \left\{ \left| c^T \sum_{1\leq j\leq d} \alpha_{ij} a_j \right| \right\}.$$

As the values $|\alpha_{ij}|$, $1 \leq i \leq m$, $1 \leq j \leq d$ are bounded by α, we obtain:

$$\omega_c(\Gamma) \leq 2\alpha \sum_{1\leq j\leq d} |c^T a_j|.$$

Let A denote the matrix whose rows correspond to a_j^T, $1 \leq j \leq d$; we can rewrite the previous expression:

$$\omega_c(\Gamma) \leq 2\alpha ||A^T c||_1 \leq 2\alpha d^{1/2} ||A^T c||_2. \tag{3}$$

As $\mathrm{conv}(K)$ is included in $\mathrm{conv}(\Gamma)$, we immediately know:

$$\omega(K) \leq \omega(\Gamma) \leq \omega_c(\Gamma).$$

Thus, for a given vector c, we obtain an upper bound for $\omega(K)$. For solving our problem, it is sufficient to only test the vector u in $\mathbb{Z}^{d*}$ satisfying:

$$\omega_u(K) \leq 2\alpha d^{1/2} ||A^T c||_2.$$

Let us try to determine a lower bound for the term $\omega_u(K)$. For any j, $1 \leq j \leq d$, there exist two points u_j and v_j in K such that $a_j = u_j - v_j$. As for any j, $1 \leq j \leq d$, $\mathrm{conv}(\{u_j, v_j\})$ is included in $\mathrm{conv}(K)$, we have for any $u \in \mathbb{Z}^{d*}$, $\omega_u(\{u_j, v_j\}) \leq \omega_u(K)$. Thus, by definition of the lattice width along a direction $u \in \mathbb{Z}^{d*}$, we obtain:

$$|u^T a_j| \leq \omega_u(K) \quad \text{for } 1 \leq j \leq d.$$

Then, it follows for any $u \in \mathbb{Z}^{d*}$:

$$d^{-1/2} ||A^T u||_2 \leq \omega_u(K).$$

Thus, we can conclude that it is sufficient to test only the vector u in $\mathbb{Z}^{d*}$ satisfying:

$$||A^T u||_2 \leq 2\alpha d ||A^T c||_2. \tag{4}$$

As the right term is fixed for a given c, we can compute a vector c with some interesting properties. The more natural approach is to compute the shortest vector v in the lattice given by A and to choose c such that $A^T c = v$. Hence the upper bound becomes a bound independent on the direction c and we get:

$$\|A^T u\|_2 \leq 2\alpha d \|v\|_2.$$

It follows that the set of tested vectors is contained in a ball with a radius independent of the set K and solely based on the surrounding set Γ. Now, according to Nguyen and Vidick [25], we have the following lemma.

Lemma 3 (*Lemma 3.2 in [25]*). *Let L be a lattice in $\mathbb{R}^d$ and a radius $R > 0$. Let us denote by $B_d(R)$ the ball with radius R. Then, $|L \cap B_d(R)| \leq 2^{c_R d}$ where $c_R = \log_2\left(1 + \frac{2R}{\lambda_1(L)}\right)$.*

In this lemma, $\lambda_1(L)$ is the shortest vector in L for the 2-norm. Hence, it is exactly $\|v\|_2$. As a consequence, since $R = 2\alpha\lambda_1(L)$ in our case, we immediately conclude that the number of points of L in the ball $B_d(R)$ is bounded by $2^{d\log_2(1+4\alpha d)}$. Hence, the bound is independent of K and is in fact a constant solely based on the dimension d and the bound α. Since there is a constant number of points in the ball, all points can be tested to extract the lattice width. Of course, an approximation of the shortest vectors can be used in place of v to avoid the difficulty of its computing in arbitrary dimension.

At the beginning of the section, we made the hypothesis that the computation of a particular surrounding polytope with specific properties is always possible. Indeed, these mathematical results only hold if it is actually the case. In the next section, we propose different methods to compute this polytope, whatever the dimension. Moreover, we specify the value of the constant called α depending on the method.

5. Surrounding polytope computation

Let K denote a set of n points in $\mathbb{Z}^d$. We look for a sequence of d vectors $(a_j)_{1\leq j\leq d}$ such that for any j, $1 \leq j \leq d$, there exist two points u_j and v_j of K satisfying $a_j = u_j - v_j$. From this sequence of vectors, we must be able to build a surrounding polytope $\Gamma = conv((\gamma_i)_{1\leq i\leq m})$ such that Γ contains K and such that each γ_i, $1 \leq i \leq m$, satisfies $\gamma_i = \sum_{j=1}^{d} \alpha_{ij} a_j$. An implicit

goal is to obtain the smallest possible upper bound for the $|\alpha_{ij}|$ values in order to improve the performance of the algorithm, $1 \leq i \leq m$, $1 \leq j \leq d$. We show afterwards that, in the two-dimensional case, we can compute such a surrounding polygon in $O(n)$ time. The corresponding approaches become too difficult to extend to the three-dimensional case. So, we present a simpler and more efficient approach that can be used in any dimension: a greedy approach.

5.1. Deterministic approaches

5.1.1. Existing approach

We first find in the literature the work of Fleischer et al. [13] that confirms that such a polytope exists. We recall this result in the two-dimensional case:

Theorem 1. *For any convex body P, let t denote a greatest area triangle contained in P. The convex body P is contained in the dilatation of t by an expansion factor of at most $9/4$.*

From [18], we know that the convex hull of the set of integer points in the plane can be computed in linear time. Thus, we can operate on $conv(K)$ without damaging the overall time complexity. Moreover, we know from [8] that there exists a greatest area triangle included in K whose vertices correspond to vertices of K. So, let T denote the dilatation of a maximal area triangle $t = ABC$ of K by a factor of $9/4$. The triangle T corresponds to a solution of our surrounding polygon problem. Indeed, if we set the origin at the point A, its three vertices are of the form $\alpha_{i1}AB + \alpha_{i2}AC$ where $|\alpha_{i1}|$ and $|\alpha_{i2}|$ bounded by 1 for $1 \leq i \leq 3$. So, the first approach that we propose consists in computing a maximum area triangle of K and then consider its dilatation with a factor $9/4$. Boyce et al. propose an overall method to compute a greatest area k-gon which runs in $O(kn \log n + n \log^2 n)$ time and in $O(n \log n)$ time when $k = 3$ (see [3]). Dobkin et al. focus on the two-dimensional case and they show that the computation of a greatest area triangle runs in linear time performing at most $10n$ triangle area computations (see [8]). Their method proceeds in two steps. First, a rooted triangle of maximal area is computed in linear time. This means that one of the vertices is kept fixed during the construction. So, for a fixed vertex v, two vertices a and b are determined such that the triangle abv has a maximal area. Then, any triangle with greatest area must be *interleaved* with the triangle abv. Two polygons with vertices in a given collection of points are said to interleave, if between every two successive vertices of one, there is a vertex of the other (possibly coinciding with one of them) [3]. Hence, the vertices of a rooted triangle with maximal area are moved in clockwise order to determine a triangle with maximal area. Unfortunately, even if the construction can be extended to the 3d case as in [24], this does not lead to an efficient algorithm since from a rooted tetrahedron of maximal volume it is now difficult to obtain an efficient ordering of the search for a non-rooted tetrahedron of maximal volume.

5.1.2. Another approach using elementary geometry

We introduce a simpler approach to compute a surrounding polygon that only requires $2n$ distance computations. Moreover, it does not need the computation of the convex hull of K. This method consists in computing a surrounding parallelogram. Let A and B denote the leftmost and the rightmost point of K respectively according to the x-axis. Let N denote the normal vector of AB with positive y-coordinate. Let C and D denote the extremal point of K in the direction N and $-N$ respectively. We show afterwards that the set K is contained in a parallelogram of sides CD and $2AB$. This parallelogram corresponds to a solution of our problem because its vertices can be expressed as $\alpha_1 CD + \alpha_2 AB$ where $|\alpha_1|$ and $|\alpha_2|$ do not exceed 1 if the origin is well chosen.

Let $EFGH$ denote the parallelogram bounded by the two vertical straight lines passing through A and B and bounded by the two straight lines of direction vector AB passing trough C and D. By construction, the set K is included in the parallelogram $EFGH$, but the vertical sides of this parallelogram cannot be expressed using vectors rooted in points of K. As a result, we try to minimize δ such that $EFGH$ is contained in a parallelogram of side CD and of side δAB. We show that δ equals 2. Indeed, the points A and B may not be extremal according to the normal vector of CD and in this case, δ is strictly greater than 1 (see Fig. 3.a). The "worst case" happens when the points C and D coincide with opposite vertices of the parallelogram $EFGH$ as in Fig. 3.b. In this case, we notice that half of the parallelogram $EFGH$ is included in a parallelogram of side AB and CD. As a consequence, it is sufficient to double the side AB of the parallelogram such that the new parallelogram becomes large enough.

Even if these two methods run in linear time, they are not easily extensible to the three-dimensional case. As a result, we describe in the next section a greedy method to compute a surrounding parallelepiped in the three-dimensional case. This method can be extended to higher dimensions but we focus on the two-dimensional and on the three-dimensional case for convenience of presentation.

5.2. A greedy approach

We first introduce the definition of a *maximal growing triangle* and of a *maximal growing tetrahedron*. We show afterwards how to compute such a maximal growing simplex and how to deduce a surrounding polytope with the requested properties.

5.2.1. Definitions

Definition 1. Let S denote a two-dimensional set of points. A triangle T whose vertices belong to S is a *maximal growing triangle* relative to S if for each edge e of T, the furthest point from e in the set S corresponds to a vertex of T.

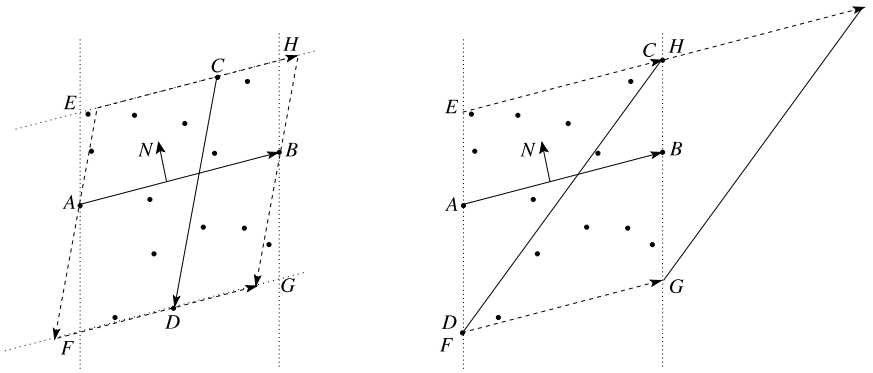


Fig. 3. Construction of the parallelograms.

Definition 2. Let K denote a three-dimensional set of points. A tetrahedron G whose vertices belong to K is a *maximal growing tetrahedron* relative to K if for each face f of G, the furthest point from f in the set K corresponds to a vertex of G.

We first notice that the vertices of a maximal growing triangle or tetrahedron obviously correspond to vertices of the convex hull of the initial set of points. We can also notice that a maximal growing triangle or a maximal growing tetrahedron does not always correspond to a greatest area triangle or to a greatest volume tetrahedron respectively. Indeed, given a maximal growing triangle T relative to a set of points S in $\mathbb{Z}^2$, a greatest area triangle relative to S corresponds either to T or to a triangle which interleaves with T in the vertices of the convex hull of the set of points. Fig. 4 shows an example: here, the triangle T corresponds to a maximal growing triangle which is not of greatest area whereas the dashed triangle T', which interleaves with T on the convex hull of the set of points, is of greatest area.

5.2.2. Algorithm design

We describe in this section our greedy but efficient in practice approach to compute a maximal growing tetrahedron G. Moreover, this method easily extends to the d-dimensional case, contrary to the previous approaches. Let $(f_l)_{1\leq l\leq 4}$ and $(v_l)_{1\leq l\leq 4}$ denote the faces and the vertices of G respectively such that the vertex v_l is opposite to the face f_l for $1 \leq l \leq 4$. We first initialize G using four non-coplanar points of K. Our method consists in "pushing" the vertices of G among the points of K until G corresponds to a maximal growing tetrahedron. Repeatedly, for each face f_l of G, the algorithm tries to grow the tetrahedron G by moving the current vertex v_l to a point of K which is the furthest point from f_l. The method stops when no more growing can be done. This approach always finds a valid solution because at each step the volume of the current tetrahedron increases at least by one and this value is bounded by the volume of $conv(K)$. An overview of the method follows:

```
MAXIMAL GROWING TETRAHEDRON ALGORITHM:
Entries: K = {k_1, k_2, ..., k_n}, G = (v_1, v_2, v_3, v_4)
1 DO
2    STABLE ← true;    l ← 1;
3    WHILE STABLE AND l ≤ 4
4        IF (FurthestPoint(K, f_l) = v_l)
5           l ← l + 1
6        ELSE
7           STABLE ← false;
8           v_l ← FurthestPoint(K, f_l);
9 UNTIL STABLE = true
```

For convenience of presentation, we illustrate the algorithm with an example in the plane. Fig. 5 shows the successive triangles computed by our algorithm until stability.

This algorithm achieves an $O(n^{d+1})$ time complexity where n corresponds to the number of points in the set and d corresponds to the dimension we consider. However, as for many greedy approaches, we can highlight the efficiency in practice of our method with experimental results. It proves that this algorithm can be used in practice in spite of its high time complexity.

The main criterion of efficiency seems to be the number of different triangles which are created during the running of our algorithm until stability. As a consequence, we compute for sets of points of various cardinalities the maximal number of different triangles computed during the running of our algorithm and the number of different triangles computed in the mean. The set of points are randomly generated. We notice on Fig. 6 that the number of generated triangles does not depend very much of the cardinality of the set of points. Indeed, for a set of 10 points, about 3.49 triangles are computed in the

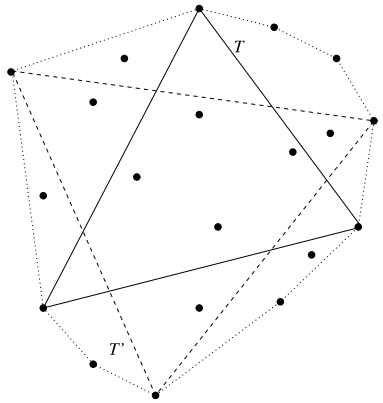


Fig. 4. Maximal growing triangle T and greatest area triangle T'.

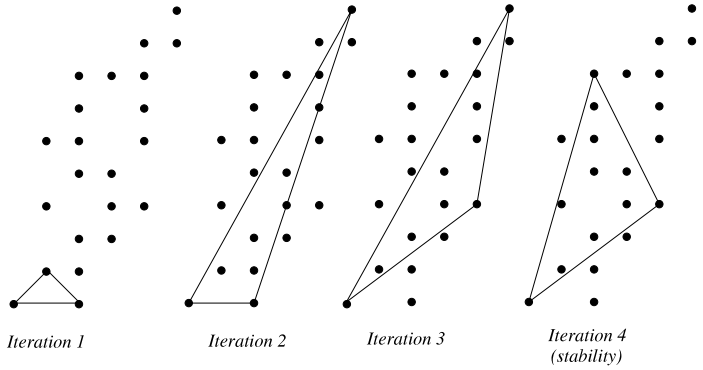


Fig. 5. Example of running of the maximal growing triangle algorithm.

number of points	*number of triangles (in the mean)*	*number of triangles (maximum)*
10	*3.49*	*7*
100	*4.62*	*10*
1000	*5.03*	*12*
10 000	*5.2*	*13*
100 000	*5.42*	*13*

Fig. 6. Number of generated triangles.

number of points	*number of tetrahedra (in the mean)*	*number of tetrahedra (maximum)*
10	*3.20*	*8*
100	*5.12*	*14*
1000	*5.94*	*12*
10 000	*6.06*	*11*
100 000	*6.125*	*12*

Fig. 7. Number of generated tetrahedra.

mean as against 5.42 triangles for 100,000 points. In the three-dimensional case, we compare in the same way the number of created tetrahedra during the running of the method as shown in Fig. 7. The experimental results show that about 6 different tetrahedra are computed given a set of 100,000 points.

number of points	*average running time (ms) (without improvement)*	*average running time (ms) (with improvement)*
100	*0.031*	*0.036*
1000	*0.314*	*0.272*
10 000	*3.422*	*2.929*
100 000	*31.206*	*28.236*

Fig. 8. Running times of our method in the planar case.

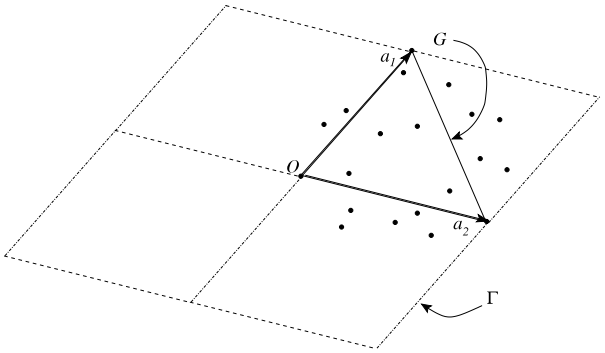


Fig. 9. Example of surrounding parallelogram.

At each iteration, either to compute a new triangle or tetrahedron or to prove the stability of the current one, we need to compute the distance of each point of the initial set relative to an edge of the current simplex. In order to limit the number of distance computations, we could eliminate some points of the considered set of points. Indeed, the vertices of the successive computed triangles correspond to vertices of the convex hull of the set of points. As a consequence, at each iteration, the points located in the interior of the current triangle or tetrahedron will never correspond to vertices of the computed triangle or tetrahedron at a later iteration. So, they could be eliminated. However, deciding if a point is located in the interior of a triangle or of a tetrahedron is a little costly and such a test should be done for each point of the current set of points. We compare the running times of our algorithm in the planar case without any improvement and with this last improvement. We notice in Fig. 8 that its benefit is almost insignificant.

5.2.3. Computation of a surrounding polytope

We show afterwards that we can compute a surrounding polytope relative to the set of points K such that its vertices γ_i, $1 \leq i \leq m$, satisfy $\gamma_i = p + \sum_{1\leq j\leq d} \alpha_{ij} a_j$ where $|\alpha_{ij}|$ is bounded by 1 when the origin is well chosen. This surrounding polytope is inferred by the computation of a maximal growing simplex. We describe the computation of a surrounding parallelepiped using a maximal growing tetrahedron in the three-dimensional case. Such a construction can be done in the same way in the planar case or in higher dimension.

We recall that $(f_l)_{1\leq l\leq 4}$ and $(v_l)_{1\leq l\leq 4}$ denote the faces and the vertices of a maximal growing tetrahedron G respectively such that the vertex v_l is opposite to the face f_l for $1 \leq l \leq 4$. Let us move the origin O to v_1; this transformation is always possible because the lattice width is invariant under translation. Let $(a_j)_{1\leq j\leq 3}$ denote the three vectors defined by $v_{j+1} - v_1$ where $(v_j)_{1\leq j\leq 4}$ denote the vertices of a maximal growing tetrahedron G of K. We show that the set K is contained in a parallelepiped Γ whose vertices $(\gamma_i)_{1\leq i\leq 8}$ are of the form $\gamma_i = \sum_{1\leq j\leq 3} \alpha_{ij} a_j$ where each $|\alpha_{ij}|$ is bounded by 1. As the maximal growing tetrahedron we use is non-flat by definition, any point k_l of K can be written as $k_l = O + \sum_{1\leq j\leq 3} \delta_{lj} a_j$. Let us show that we have $|\delta_{ij}| \leq 1$ for $1 \leq i \leq n$, $1 \leq j \leq d$. Indeed, suppose that there exists an index i and a index j such that a $|\delta_{ij}|$ is strictly greater than 1. It would contradict the fact that G corresponds to a maximal growing tetrahedron because the vertex v_{j+1} would not be extremal. As a result the set K is contained in a parallelepiped Γ whose vertices $(\gamma_i)_{1\leq i\leq 8}$ are of the form $\gamma_i = \sum_{1\leq j\leq 3} \alpha_{ij} a_j$ where each $|\alpha_{ij}|$ is bounded by 1. Figs. 9 and 10 show examples in the plane and in the three-dimensional space respectively: G corresponds to a maximal growing triangle (resp. a maximal growing tetrahedron) and Γ corresponds to a surrounding parallelogram (resp. a surrounding parallelepiped).

6. Conclusion

We have described in this paper a new algorithm to compute the lattice width of a set of points K. In the planar case, it runs in linear time relative to the size of K, which is optimal. Moreover, its principle is directly extensible to an arbitrary dimension even if intermediate constructions become more complex in that case. Our greedy approach for the computation of the surrounding polytope simplifies greatly the application of this algorithm to an arbitrary dimension. Indeed, even if it

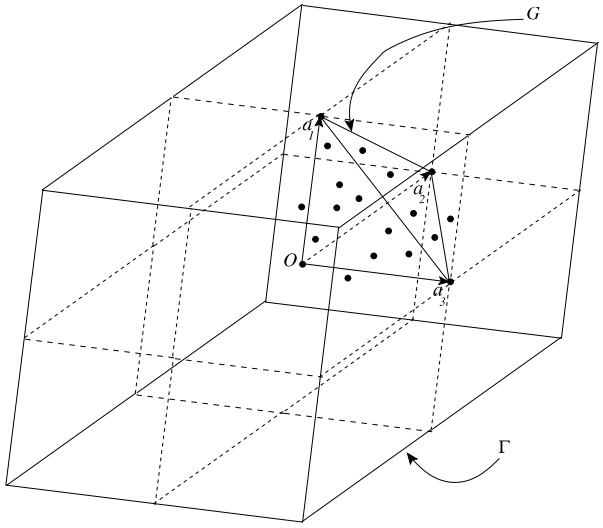


Fig. 10. Example of surrounding parallelepiped.

achieves an $O(n^{d+1})$ worst-case time complexity where n corresponds to the cardinality of the set of points and d corresponds to the dimension, this method is very efficient in practice. Moreover, it avoids the computation of an inscribed d-simplex of greatest volume which is a problem in $O(n^4)$ at least in the three-dimensional case for the best known algorithm [24]. This problem is an interesting problem in itself and could be the subject of future research since it is known that the dilation constant of this d-simplex is independent of K and thus it guarantees the smallest possible space search in any case. We also plan to extend the definition of linearity given in [12] to an arbitrary dimension. The optimal complexity of our new algorithm in the planar case is a key point in that construction.

References

[1] V.I. Arnold, Higher dimensional continued fractions, Regul. chaotic Dyn. 3 (1998) 10–17.
[2] A. Barvinok, A Course in Convexity, in: Graduates Studies in Mathematics, vol. 54, Amer. Math. Soc., 2002.
[3] J.E. Boyce, D.P. Dobkin, R.L. Drysdale, L.J. Guibas, Finding extremal polygons, in: STOC, 1982, pp. 282–289.
[4] Emilie Charrier, Lilian Buzer, Fabien Feschet, Efficient lattice width computation in arbitrary dimension, in: DGCI, 2009, pp. 46–56.
[5] W. Cook, M. Hartman, R. Kannan, C. McDiarmid, On integer points in polyhedra, Combinatorica 12 (1992) 27–37.
[6] M. de Berg, O. Schwarzkopf, M. van Kreveld, M. Overmars, Computational Geometry: Algorithms and Applications, Springer-Verlag, 2000.
[7] Isabelle Debled-Rennesson, Jean-Pierre Reveillès, A linear algorithm for segmentation of digital curves, Int. J. Pattern Recogn. Artif. Intell. 9 (4) (1995) 635–662.
[8] D.P. Dobkin, L. Snyder, On a general method for maximizing and minimizing among certain geometric problems, in: SFCS'79: Proceedings of the 20th Annual Symposium on Foundations of Computer Science, IEEE Computer Society, Washington, DC, USA, 1979, pp. 9–17.
[9] F. Eisenbrand, S. Laue, A linear algorithm for integer programming in the plane, Math. Program. Ser. A 102 (2005) 249–259.
[10] F. Eisenbrand, G. Rote, Fast 2-variable integer programming, in: K. Aardal, B. Gerards (Eds.), Integer Programming and Combinatorial Optimization, in: LNCS, vol. 2081, Springer-Verlag, 2001, pp. 78–89.
[11] Fabien Feschet, The exact lattice width of planar sets and minimal arithmetical thickness, in: R. Reulke, U. Eckardt, B. Flach, U. Knauer, K. Polthier (Eds.), IWCIA, in: Lecture Notes in Computer Science, vol. 4040, Springer, 2006, pp. 25–33.
[12] Fabien Feschet, The lattice width and quasi-straightness in digital spaces, in: 19th International Conference on Pattern Recognition, ICPR, IEEE, 2008, pp. 1–4.
[13] R. Fleischer, K. Mehlhorn, G. Rote, E. Welzl, C.-K. Yap, Simultaneous inner and outer approximation of shapes, Algorithmica 8 (5&6) (1992) 365–389.
[14] W. Harvey, Computing two-dimensional Integer Hulls, SIAM J. Comput. 28 (6) (1999) 2285–2299.
[15] D. Hirschberg, C.K. Wong, A polynomial-time algorithm for the knapsack problem with two variables, J. Assoc. Comput. Mach. 23 (1976) 147–154.
[16] M.E. Houle, G.T. Toussaint, Computing the width of a set, IEEE Trans. Pattern Anal. Mach. Intell. 10 (5) (1988) 761–765.
[17] M.E. Houle, G.T. Toussaint, Computing the width of a set, IEEE Trans. Pattern Anal. Mach. Intell. 10 (5) (1988) 761–765.
[18] A. Hübler, R. Klette, K. Voss, Determination of the convex hull of a finite set of planar points within linear time, Elektron. Inf.verarb. Kybern. 17 (2–3) (1981) 121–139.
[19] M. Kaib, C.-P. Schnörr, The generalized gauss reduction algorithm, J. Algorithms 21 (3) (1996) 565–578.
[20] R. Kannan, A polynomial algorithm for the two variable integer programming problem, J. Assoc. Comput. Mach. 27 (1980) 118–122.
[21] G. Lachaud, Klein polygons and geometric diagrams, Contemp. Math. 210 (1998) 365–372.
[22] G. Lachaud, Sails and klein polyhedra, Contemporary Math. 210 (1998) 373–385.
[23] H.W. Lenstra, Integer programming with a fixed number of variables, Math. Oper. Res. 8 (1983) 535–548.
[24] S.I. Lyashko, B.V. Rublev, Minimal ellipsoids and maximal simplexes in 3D euclidean space, Cybernet. Systems Anal. 39 (6) (2003) 831–834.
[25] P.Q. Nguyen, T. Vidick, Sieve algorithms for the shortest vector problem are practical, J. Math. Cryptol. 2 (2) (2008).
[26] J.-P. Reveillès, Géométrie discrète, calcul en nombres entiers et algorithmique, Thèse d'etat, Université Louis Pasteur, Strasbourg, France, 1991.
[27] G. Rote, Finding a shortest vector in a two-dimensional lattice modulo m, Theoret. Comput. Sci. 172 (1–2) (1997) 303–308.
[28] H.E. Scarf, Production sets with indivisibilities part I and part II, Econometrica 49 (1981) 1–32, 395–423.
[29] A. Schrijver, Theory of Linear and Integer Programming, John Wiley and Sons, 1998.